

UNIT 2: SORTING AND LINKED LIST

2.1 Selection Sort

Selection Sort is a simple sorting algorithm that repeatedly selects the smallest element from the unsorted part of an array and places it at the beginning of the unsorted part.

The array is divided into two parts:

1. **Sorted part**
2. **Unsorted part**

In every pass, the smallest element from the unsorted part is selected and swapped with the first element of the unsorted part.

Example

Consider the array:

[64, 25, 12, 22, 11]

Pass 1: Smallest element = 11

Swap 11 with 64:

[11, 25, 12, 22, 64]

Pass 2: Smallest element = 12

Swap 12 with 25:

[11, 12, 25, 22, 64]

Pass 3: Smallest element = 22

Swap 22 with 25:

[11, 12, 22, 25, 64]

Pass 4: 25 is already in the correct position.

Sorted Array:

[11, 12, 22, 25, 64]

Algorithm

1. Start from the first element.
2. Find the smallest element in the unsorted portion.
3. Swap it with the first element of the unsorted portion.
4. Move the boundary of the sorted portion one position forward.
5. Repeat until the entire array is sorted.

Advantages

- Simple and easy to understand.
- Requires very little extra memory.

- Performs fewer swaps compared to some other sorting methods.

Disadvantages

- Slow for large datasets.
- Performs many comparisons even if the array is already sorted.

Complexity

- **Best Case:** $O(n^2)$
 - **Average Case:** $O(n^2)$
 - **Worst Case:** $O(n^2)$
 - **Space Complexity:** $O(1)$
-

2.2 Bubble Sort

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order.

After each pass, the largest unsorted element moves or "**bubbles**" to the end of the array.

Example

Consider:

[5, 3, 8, 4, 2]

Pass 1:

Compare 5 and 3 → Swap

[3, 5, 8, 4, 2]

Compare 5 and 8 → No swap

[3, 5, 8, 4, 2]

Compare 8 and 4 → Swap

[3, 5, 4, 8, 2]

Compare 8 and 2 → Swap

[3, 5, 4, 2, 8]

The largest element, **8**, is now at the end.

The process continues until the array is sorted.

Sorted Array:

[2, 3, 4, 5, 8]

Algorithm

1. Compare the first two adjacent elements.
2. If the first element is greater than the second, swap them.

3. Move to the next pair of elements.
4. Continue until the end of the array.
5. Repeat the process for the remaining unsorted elements.
6. Stop when no swaps are required.

Advantages

- Very simple to implement.
- Easy to understand.
- Requires no additional data structure.
- Can be efficient for small or nearly sorted arrays.

Disadvantages

- Very slow for large datasets.
- Performs many comparisons and swaps.

Complexity

- **Best Case:** $O(n)$ with an optimized implementation
 - **Average Case:** $O(n^2)$
 - **Worst Case:** $O(n^2)$
 - **Space Complexity:** $O(1)$
-

2.3 Insertion Sort

Insertion Sort is a sorting algorithm that builds the final sorted array one element at a time.

It works similarly to arranging playing cards in your hand. Each new element is compared with the already sorted elements and inserted into its correct position.

Example

Consider:

[5, 2, 4, 6, 1]

Initially, 5 is considered sorted.

Step 1: Insert 2 before 5:

[2, 5, 4, 6, 1]

Step 2: Insert 4 between 2 and 5:

[2, 4, 5, 6, 1]

Step 3: 6 is already in the correct position:

[2, 4, 5, 6, 1]

Step 4: Insert 1 at the beginning:

[1, 2, 4, 5, 6]

Sorted Array:

[1, 2, 4, 5, 6]

Algorithm

1. Consider the first element as sorted.
2. Select the next element as the key.
3. Compare the key with elements in the sorted portion.
4. Shift larger elements one position to the right.
5. Insert the key into its correct position.
6. Repeat until all elements are sorted.

Advantages

- Simple to implement.
- Efficient for small datasets.
- Efficient for nearly sorted data.
- Requires very little extra memory.
- Stable sorting algorithm.

Disadvantages

- Slow for large and randomly arranged datasets.
- Requires many shifts in the worst case.

Complexity

- **Best Case:** $O(n)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

2.4 Introduction to Linked List

A **Linked List** is a linear data structure in which elements are stored in separate memory locations and connected using links or pointers.

Unlike an array, linked list elements do not need to be stored in contiguous memory locations.

Each element of a linked list is called a **Node**.

A node generally contains two parts:

1. **Data** – Stores the actual value.
2. **Link/Next** – Stores the address of the next node.

Structure of a Node

[DATA | NEXT]

Example:

[10 | Address of Node 2] → [20 | Address of Node 3] → [30 | NULL]

The first node is called the **Head**.

The last node points to **NULL**, indicating the end of the linked list.

Types of Linked Lists

1. **Singly Linked List**
 - Each node contains a pointer to the next node.
2. **Doubly Linked List**
 - Each node contains pointers to both the previous and next nodes.
3. **Circular Linked List**
 - The last node points back to the first node.

Advantages

- Dynamic size.
- Efficient insertion and deletion.
- Does not require contiguous memory.
- Memory is allocated as required.

Disadvantages

- Extra memory is required for pointers.
- Direct/random access is not possible.
- Searching is slower compared to arrays.
- Pointer handling makes implementation more complex.

2.5 Representation of Linked List in Memory

In a linked list, nodes can be stored anywhere in the available memory. They are connected using pointers.

Consider the following linked list:

10 → 20 → 30 → NULL

A possible memory representation is:

Node	Address	Data	Next
------	---------	------	------

Node 1	1000	10	2500
--------	------	----	------

Node 2	2500	20	1800
--------	------	----	------

Node 3	1800	30	NULL
--------	------	----	------

The **HEAD** pointer contains the address of the first node:

HEAD = 1000

The linked list is accessed as:

HEAD → 1000 → 2500 → 1800 → NULL

The nodes do not have to be stored next to each other in memory.

Node Representation

A node can be represented as:

[Data | Next]

For example:

HEAD → [10 | 2500] → [20 | 1800] → [30 | NULL]

Important Points

- **HEAD** stores the address of the first node.
- **DATA** stores the actual value.
- **NEXT** stores the address of the next node.
- **NULL** indicates the end of the linked list.

2.6 Searching a Linked List

Searching a linked list means finding a particular element or key in the list.

Since linked lists do not provide direct access to elements, searching is generally performed using **linear search**.

Example

Consider:

10 → 20 → 30 → 40 → NULL

Suppose we want to search for **30**.

The process is:

Step 1: Start at HEAD → 10

10 ≠ 30

Step 2: Move to next node → 20

20 ≠ 30

Step 3: Move to next node → 30

30 = 30

Element Found

Algorithm

1. Start from the HEAD node.
2. Compare the data of the current node with the search key.
3. If the data matches, the element is found.
4. Otherwise, move to the next node.
5. Continue until the element is found or NULL is reached.
6. If NULL is reached, the element is not present.

Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(n)$
- **Worst Case:** $O(n)$

Where **n** is the number of nodes.

2.7 Memory Allocation and Garbage Collection

Memory Allocation

Memory allocation is the process of assigning memory space to data during program execution.

In linked lists, memory is generally allocated dynamically because the size of the list may change during program execution.

When a new node is required:

1. The program requests memory.
2. A memory block is allocated.
3. Data is stored in the new node.
4. The node is connected to the linked list using pointers.

Dynamic Memory Allocation

Dynamic memory allocation allows memory to be allocated during runtime.

Common functions in C include:

- malloc()
- calloc()
- realloc()
- free()

The free() function releases dynamically allocated memory.

Garbage Collection

Garbage Collection is the automatic process of identifying and releasing memory that is no longer being used by a program.

When a node is deleted from a linked list, its memory should be released. If the memory is not released, it may become unused but still occupy memory. This condition is called a **memory leak**.

Languages such as **Java, Python, and C#** provide automatic garbage collection.

Example

Suppose a linked list is:

10 → 20 → 30 → NULL

If node 20 is deleted:

10 → 30 → NULL

The memory previously occupied by node 20 is no longer required.

A garbage collector can identify that the old node is no longer accessible and reclaim its memory.

Difference Between Memory Allocation and Garbage Collection

Memory Allocation	Garbage Collection
Provides memory to a program	Reclaims unused memory
Used when new data is created	Used when data is no longer required
Can be manual or automatic	Usually automatic in managed languages
In C, functions like malloc() are used In Java and Python, automatic garbage collection is available	

2.8 Insertion and Deletion in Linked List

Insertion in Linked List

Insertion means adding a new node to a linked list.

Insertion can be performed at:

1. Beginning
2. End
3. Specific position

A. Insertion at the Beginning

Consider the original list:

10 → 20 → 30 → NULL

Suppose we want to insert **5** at the beginning.

Steps

1. Create a new node containing 5.
2. Set the new node's NEXT pointer to HEAD.
3. Update HEAD to point to the new node.

Result:

5 → 10 → 20 → 30 → NULL

Advantage

Insertion at the beginning is very fast.

Time Complexity: $O(1)$

B. Insertion at the End

Consider:

10 → 20 → 30 → NULL

Insert **40** at the end.

Steps

1. Create a new node containing 40.
2. Traverse the list until the last node is reached.
3. Set the last node's NEXT pointer to the new node.
4. Set the new node's NEXT pointer to NULL.

Result:

10 → 20 → 30 → 40 → NULL

Time Complexity

O(n) when only a HEAD pointer is available.

C. Insertion at a Specific Position

Suppose the list is:

10 → 20 → 40 → NULL

We want to insert **30** between 20 and 40.

Steps

1. Create a new node containing 30.
2. Find the node containing 20.
3. Set the new node's NEXT pointer to the node containing 40.
4. Set the NEXT pointer of node 20 to the new node.

Result:

10 → 20 → 30 → 40 → NULL

Deletion in Linked List

Deletion means removing a node from a linked list.

Deletion can be performed from:

1. Beginning
 2. End
 3. Specific position
-

A. Deletion from the Beginning

Original list:

10 → 20 → 30 → NULL

Delete the first node.

Steps

1. Move HEAD to the second node.
2. Release the memory of the old first node.

Result:

20 → 30 → NULL

Time Complexity

O(1)

B. Deletion from the End

Original list:

10 → 20 → 30 → NULL

Delete the last node.

Steps

1. Traverse the list to find the second-last node.
2. Set its NEXT pointer to NULL.
3. Release the memory of the last node.

Result:

10 → 20 → NULL

Time Complexity

O(n) when only a HEAD pointer is available.

C. Deletion of a Specific Node

Original list:

10 → 20 → 30 → 40 → NULL

Suppose we want to delete **30**.

Steps

1. Find the node containing 30.
2. Find the previous node containing 20.
3. Change the NEXT pointer of 20 to point to 40.
4. Release the memory of the node containing 30.

Result:

10 → 20 → 40 → NULL

Time Complexity

O(n) in the general case because the node may need to be located first.

Difference Between Array and Linked List

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Size	Usually fixed	Dynamic
Access	Direct/Random	Sequential
Access Time	$O(1)$	$O(n)$
Insertion	Costly in middle	Easier
Deletion	Costly in middle	Easier
Memory	No pointer overhead	Extra memory for pointers
Searching	Faster with direct access	Requires traversal
Memory Allocation	Usually static or fixed-size	Dynamic

Conclusion

Sorting algorithms are used to arrange data efficiently. **Selection Sort, Bubble Sort, and Insertion Sort** are basic sorting techniques with different approaches to arranging elements. A **Linked List** is a dynamic linear data structure made up of nodes connected using pointers. It provides efficient insertion and deletion compared to arrays but does not support direct access to elements. Understanding memory allocation, garbage collection, searching, insertion, and deletion is essential for effectively working with linked lists.